

tant one. Whenever an application has external dependencies which are required for it to successfully build and run, they need to be part of the source control! Otherwise those dependencies would be missing from the project source of all other members and their builds and runs will fail. This can also break deployment automations and test suites. Usually a dependency management tool like maven or gradle is used as an easier alternative to committing all dependencies to project source repo.

Integrated Development Environments (IDEs)

The bread and butter (or the knife really) of almost every developer involved in designing an application, Integrated Development Environments or IDEs as they are commonly known are powerful tools that act as a centralized one stop solution for coding requirements. They feature a language specific editor, function listing, project structure navigation, allow programs to run within their scope, as well as support multiple plug-ins to connect to source code repositories, databases or even issue tracker software. While the choice of a good repository would depend on the kind of architecture the application has, the choice of programming language used as well as the developer's preference there are some common tips around the best use of these tools to achieve better results.

Use Integrations

The objective of using an IDE first and foremost is the ease with which it allows the seamless integration of different components of the development, compilation, testing and debugging cycle. Using and leveraging these integrations is then the primary guideline while using IDEs yet surprisingly a large number of developers tend to only use an IDE to write and compile code (sometimes even compilation is done externally) while testing and debugging are done externally via other tools or command line. This actually defeats the purpose of using the IDE. We should always strive to setup and use the most number of possible integrations from within the IDE itself. These may include source code repository integration, debugging from an embedded server or virtual machine etc.

Save actions on editors

Ever had an unnecessary import dangling in your java file? These unnecessary imports slow down the application. Often when we are working on a function we import some dependency that we later realize we do not